

PLANT AI: CROP CARE AI-POWERED CROP DISEASE DETECTION

A Human-Computer Interaction and AI Systems Design Study

Bright Blankson
Final Year Project — UX/UI Design
2025

Abstract

Crop Care is a mobile application that deploys AI-based image classification to identify crop diseases from photographs of affected leaves, returning a diagnosis, a calibrated confidence score, and structured treatment guidance to the user within seconds. This paper presents the design rationale, interface architecture, and technical framing of the system, with particular attention to five problems that sit at the intersection of human-computer interaction, machine learning deployment, and computational cognition: how probabilistic uncertainty is communicated to non-technical users in a high-stakes decision context; the constraints imposed by on-device inference on low-specification hardware; the design of input acquisition systems that produce AI-usable data from users with no understanding of what the model requires; the data architecture of the History screen as a foundation for future model retraining and community-contributed annotation; and the domain shift problem that arises when a model trained on globally available datasets is deployed in a West African agricultural context with substantially different visual and biological characteristics. The application was prototyped in Figma as a final year project, covering the complete user journey from onboarding through disease detection to scan history.

Keywords: *AI uncertainty communication, on-device inference, TensorFlow Lite, Core ML, human-in-the-loop systems, input quality design, dataset bias, domain shift, active learning, HCI, agricultural AI, West Africa.*

1. Introduction

Crop Care is a mobile application designed to identify crop diseases from photographs of affected plant leaves using AI image classification, returning a diagnosis with a confidence score and structured treatment guidance. The application targets smallholder farmers in Ghana and West Africa — users who have no agricultural training, who may be using a smartphone application for the first time, and who are operating in field conditions where mobile data connectivity cannot be guaranteed. The core technical challenge is not the classification task itself, which is well-studied in the plant pathology AI literature (Mohanty, Hughes, & Salathé, 2016). The more interesting and less-solved problem is the human side of the system: how to acquire high-quality inputs from users who do not understand what the model needs; how to communicate probabilistic outputs to users who do not have a concept of model confidence; and how to deploy a capable model under the memory, compute, and connectivity constraints of a low-specification Android device in a rural field.

This paper presents the design of the Crop Care system with those five problems as its primary analytical frame. The interface decisions described are not aesthetic choices — they are responses to specific technical constraints imposed by the AI pipeline, the deployment environment, and the cognitive characteristics of the target user. The paper is intended as a bridge document between the UX/UI design of the application and the ML systems and HCI research questions that the design implicitly engages.

2. Problem and Deployment Context

2.1 The Decision Environment

The farmer using Crop Care is making an agricultural management decision under uncertainty: a plant on their farm shows symptoms of disease, and they must decide whether to treat it, what to treat it with, and how urgently. This is a structured decision problem in which the quality of the outcome — harvest preserved or lost — is highly sensitive to both the accuracy of the diagnosis and the speed with which the farmer acts on it. The Food and Agriculture Organization of the United Nations estimates that plant diseases cause losses of between 20 and 40 percent of global food production annually. For a smallholder farming five hectares or fewer, an undiagnosed disease spreading across a maize plot for three additional days while the farmer awaits a confirmation from an extension officer is not an abstract risk — it is a concrete and potentially catastrophic economic event.

The traditional information pathway — calling an extension officer, visiting an agro-dealer, posting to a community WhatsApp group — has latency measured in days and reliability that is structurally low. Agricultural extension services in Ghana operate at ratios of one officer per several thousand farmers in many districts. The information available at that latency is often generic, based on what the farmer can describe verbally rather than what the farmer can show visually, and filtered through the commercial incentives of whoever is providing it. Crop Care is designed to reduce the latency of a disease diagnosis to under a minute and to ground it in the visual evidence of the actual leaf rather than a verbal description of symptoms. The design problem is how to make that technically possible

given the constraints of the deployment environment, and how to make it cognitively usable for the people it is intended to serve.

2.2 Why Existing Tools Are Insufficient

Several AI-based plant disease identification tools exist, including PlantNet, PlantVillage Nuru, Agrio, and Plantix. A competitive audit conducted across six applications identified a common pattern: these tools are built for users in developed markets with high digital literacy, reliable connectivity, and crops relevant to European or North American agricultural contexts. PlantNet is primarily designed around European flora and performs poorly on West African crop varieties. PlantVillage Nuru was developed for East African crops and does not cover the disease strains prevalent in Ghana. None of the tools reviewed provided output in Ghanaian local languages — Twi, Ga, Hausa, or Dangme — making them functionally inaccessible to a large proportion of the intended users regardless of the quality of their AI.

More significantly from a systems design perspective, none of the tools reviewed addressed the input quality problem: the fact that the accuracy of the AI's output is directly dependent on the quality of the photograph submitted, and that farmers without guidance will consistently take photographs that fall outside the distribution of images the model was trained on. This input quality gap — the mismatch between what a non-technical user naturally produces and what the AI requires — is one of the central design problems that Crop Care was built to address.

3. System Overview

3.1 Architecture

Crop Care connects three functional components: an input acquisition system (the camera flow), an inference engine (the AI classification model), and an output presentation system (the result screen and history log). In the current prototype, the inference engine is represented as a server-side process; in the production architecture proposed in Section 7, it migrates to an on-device model running under TensorFlow Lite or Core ML to eliminate the connectivity dependency. The three components are connected through a data pipeline that begins with image capture, preprocesses the image for model input, runs inference, and returns a structured result object containing the predicted class label, the confidence score, and the associated treatment content for that disease class.

The application covers five screen groups in the prototype: Onboarding, Camera and Home, Analyzing and Result, History, and Language. Each screen group maps to a distinct phase of the user's interaction with the AI system: preparation, input acquisition, processing and result delivery, record review, and localisation. The design decisions within each screen group are described in Section 4 with explicit reference to the technical and cognitive constraints they address.

3.2 Target Hardware and Operating Environment

The primary deployment target for Crop Care is a mid-to-low-specification Android smartphone — a Tecno, Infinix, or Itel device in the GHS 400–900 price range, which represents the dominant smartphone segment in the Ghanaian market. These devices typically run Android 10 or 11, carry 2–3 GB of RAM, and have processors with limited support for hardware-accelerated neural network inference. They are used in outdoor field conditions, in variable lighting from direct sunlight to overcast, and frequently without a reliable mobile data connection. Any production version of Crop Care must function within these constraints without degradation of the core identification capability.

4. Interface Design: Five Technical Problems

4.1 Communicating AI Uncertainty to Non-Technical Users

The most consequential single design decision in Crop Care is the confidence score displayed on the result screen. The score — presented as a percentage alongside the disease name, reading 92% in the prototype — is the application's primary mechanism for communicating to the user that the AI's output is probabilistic, not certain, and that different diagnoses carry different levels of reliability. This is a non-trivial communication problem. The user base for Crop Care includes farmers who have not previously encountered the concept of a machine learning model and who are likely to interpret any output from a smartphone application as either correct or broken. The confidence score is designed to create a third interpretive state: the system is working and it has a view, but the strength of that view varies, and the user should factor that variation into their decision.

The design challenge is that a raw softmax probability is not directly interpretable by a non-technical user. A score of 0.92 from a poorly calibrated model may reflect less actual certainty than a score of 0.78 from a well-calibrated one, because softmax outputs are not reliably calibrated to match true posterior probabilities without post-hoc techniques such as temperature scaling (Guo et al., 2017). The interface presents the score as a percentage, which is the format most familiar to users from everyday contexts such as battery level and test results, rather than as a decimal probability or a categorical label. The score is displayed visually as a circular progress indicator alongside a color coding — green for high confidence, amber for medium, red for low — that allows users who cannot read the number to still extract the qualitative signal.

The deeper issue, which the interface alone cannot resolve, is that the displayed score is only meaningful if the model is calibrated. A model that returns 92% confidence on a misclassified image is more harmful than a model that returns 60% on the correct class, because it provides false assurance to a user who has been trained by the interface to treat high scores as reliable. This is a known problem in deployed classification systems, and it is one of the reasons that model calibration — not just accuracy — must be a first-class evaluation metric in the technical development of Crop Care. The design creates the communication infrastructure; the accuracy of the signal it carries depends on the ML pipeline.

The result screen also includes a Retake button, which serves a dual function: it gives the user an explicit action path when the result does not match their expectation, and it

implicitly communicates that the AI can be wrong and that disagreement is a valid response. This is an important affordance in a high-stakes decision context, where a user who mistrusts a result but sees no mechanism to challenge it may either act on a wrong diagnosis or abandon the tool entirely.

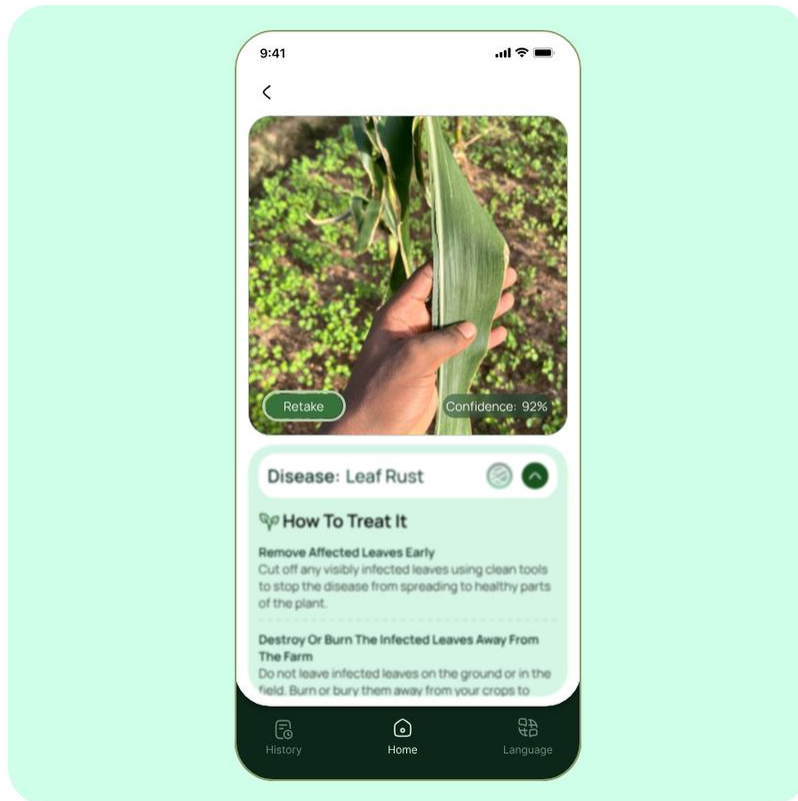


Figure 1: Result screen — confidence score display, structured treatment guidance, and user override affordance.

4.2 On-Device Inference Under Hardware Constraints

The use case for Crop Care — a farmer in a field examining a diseased plant — is precisely the scenario in which server-side inference is least viable. Rural farming areas in Ghana have limited and intermittent mobile data coverage, and the latency of a round-trip to a remote inference server in poor connectivity conditions is incompatible with the responsive, low-friction experience the interface is designed to deliver. The production architecture must implement on-device inference, running the classification model locally on the device without a network connection.

TensorFlow Lite (TFLite) and Apple's Core ML are the two primary frameworks for on-device neural network inference on Android and iOS respectively. Both support deployment of quantized convolutional neural network models at significantly reduced memory and compute cost relative to full-precision models. The standard pipeline involves training a classification model — a MobileNetV2 or EfficientNet-B0 architecture is typical for plant disease classification tasks given their favorable accuracy-to-parameter-count ratio — on a disease image dataset, then applying post-training quantization to convert

the model weights from 32-bit floating point to 8-bit integer representation. This quantization process typically reduces model size by 75% and inference latency by 30–50% on ARM processors, with a modest degradation in accuracy that must be evaluated against the accuracy requirements of the specific application.

For Crop Care's target hardware — a 2–3 GB RAM Android device without a dedicated neural processing unit — a quantized MobileNetV2 model for a classification task covering 20–30 disease classes is estimated to run inference in 150–300 milliseconds per image, which is within the range that the analyzing screen's loading indicator is designed to cover. The analyzing screen displays the captured image during processing, giving the user a moment to confirm they submitted the correct photograph. This is not merely a UX affordance — it also creates a natural pause of sufficient duration to accommodate inference latency without exposing the user to a blank loading state that would suggest the application has frozen.

The constraint that creates the most significant design tension is memory. A quantized MobileNetV2 model for a 30-class disease classification task occupies approximately 3–5 MB of storage, which is acceptable. However, the full inference pipeline — model loading, image preprocessing, buffer allocation, and output postprocessing — consumes working memory that competes with the rest of the application on a device with 2 GB RAM shared across the operating system and all running processes. The application's memory architecture must be designed to load the model at startup and retain it in memory for the duration of the session, rather than reloading it on each inference call, to avoid the latency and GC pressure of repeated model loading.

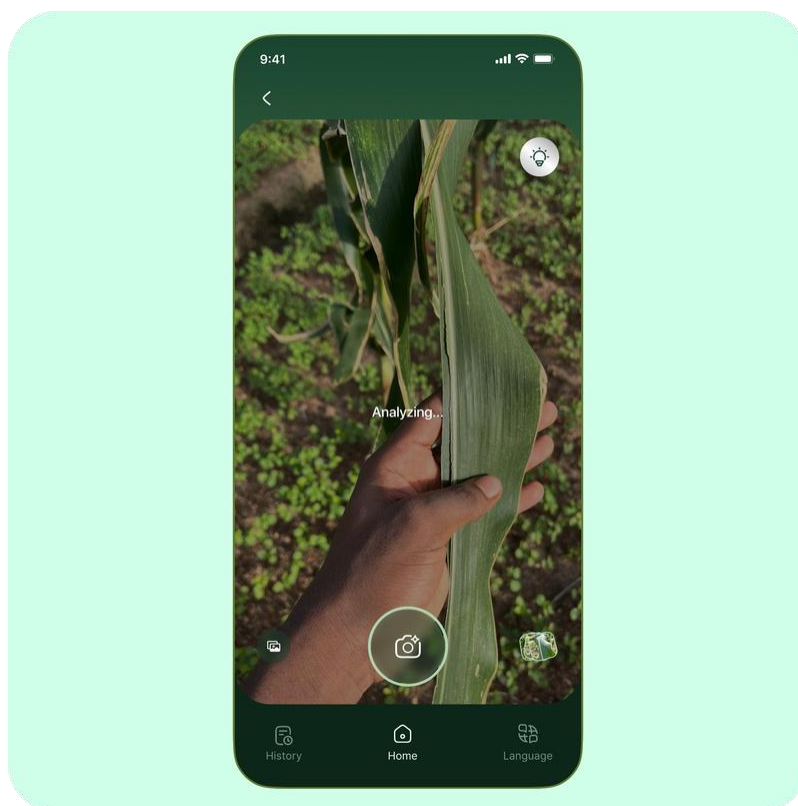


Figure 2: Analyzing screen — the processing state, designed to accommodate on-device inference latency while maintaining user orientation.

4.3 Designing Input Acquisition for AI-Usable Data

The accuracy of any image classification system is bounded not only by the quality of the model but by the quality of the inputs it receives. A well-trained model deployed in a production system where users consistently submit out-of-distribution inputs will perform substantially worse than its benchmark accuracy implies. For Crop Care, the primary sources of input degradation are motion blur from camera shake, insufficient lighting in field conditions, incorrect framing (the leaf does not fill a sufficient proportion of the frame), and occlusion (the diseased region is partially hidden by other leaves or the user's hand). These are not edge cases — they are the natural behavior of a non-technical user who has never been told what the model needs.

The camera screen addresses this through an embedded guidance system: two short, numbered instructional prompts displayed at the top of the viewfinder that guide the user on framing and lighting before they take the photograph. The prompts are not part of the onboarding flow — they are embedded in the camera view itself, ensuring that they are visible at the exact moment of image capture regardless of whether the user engaged with onboarding. This is a just-in-time instruction design, grounded in research showing that instructions presented at the moment of action are more likely to be followed than instructions presented earlier in a session (Sweller, 1988).

The camera screen went through three design iterations. The first iteration presented an unguided viewfinder with no instructions. Informal testing confirmed the expected result: users held the device at an angle that captured more background than leaf, in lighting conditions that introduced significant noise into the image. The second iteration introduced a longer text guide above the viewfinder, but the instruction block was tall enough to occlude the lower portion of the viewfinder on smaller screens, creating a conflict between the guidance and the action it was supposed to support. The final iteration reduced the guidance to two single-line prompts in a semi-transparent overlay that sits above the live camera feed without obscuring it. This design is informed by the overlay guidance pattern used in document scanning applications such as Adobe Scan and Microsoft Lens, where real-time framing cues are embedded directly in the capture view.

A future version of the camera screen should implement real-time image quality feedback — analyzing the live camera feed for blur, lighting level, and framing adequacy before the user taps the capture button, and providing dynamic feedback if the current frame is likely to produce a low-quality input. This kind of pre-capture quality gating is technically feasible with lightweight, fast-running heuristics (sharpness via Laplacian variance, brightness via mean luminance) that can run in parallel with the camera preview without impacting capture latency. It would substantially reduce the rate of inference calls on inputs that the model cannot reliably classify, and would improve the correlation between the model's confidence score and its actual accuracy — since low-quality inputs are a primary driver of miscalibration.

[SCREENSHOT]

Insert the Camera screen here — the full-screen viewfinder with the two-line guidance prompts visible at the top, the Take Photo and Choose From Gallery buttons, and the bottom navigation bar, as they appear in your Figma file.

Figure 3: Camera screen — embedded just-in-time input guidance designed to reduce out-of-distribution image submissions.

4.4 History as a Data Architecture for Model Retraining

The History screen logs every scan the user has performed, displaying the date, disease name, crop type, confidence score, and a thumbnail of the scanned image in chronological order. From a user experience perspective, the History screen serves as a personal diagnostic record. From a systems architecture perspective, it is a structured data collection layer that accumulates the inputs required for model retraining and domain adaptation.

The dataset problem in agricultural AI is well-documented. The PlantVillage dataset — the most widely used benchmark for plant disease classification research — contains 54,306 images across 38 disease classes, but the images were photographed in controlled laboratory conditions with uniform backgrounds, consistent lighting, and crop varieties grown under North American and European conditions (Hughes & Salathé, 2015). A model trained on PlantVillage and deployed in a Ghanaian field environment encounters a domain shift between the training distribution and the deployment distribution that substantially degrades its generalization performance. The inputs it receives — photographs taken in variable outdoor lighting, against complex natural backgrounds, of West African maize and cassava varieties — are systematically unlike the images it was trained on.

Addressing this domain shift requires field-captured images from the deployment context, annotated with ground-truth disease labels. The History screen creates the infrastructure for this data collection in two ways. First, it stores the raw image associated with each scan alongside the model's predicted label and confidence score, creating a set of weakly labeled training examples — examples whose labels come from the model rather than from a human expert, which are noisy but volumetrically scalable. Second, it creates a mechanism through which a user who disagrees with a diagnosis — who taps Retake after receiving a result — generates an implicit negative label for the submitted image and class pair, which is a weak negative signal useful for identifying the failure modes of the current model.

A future version of the system should implement a voluntary annotation pathway in which users can confirm or correct a diagnosis after it has been field-verified — for example, by confirming after treatment whether the prescribed intervention was effective. Confirmed diagnoses create hard-labeled positive examples. Corrections — where the user indicates the model was wrong and provides the correct disease name — create hard-labeled negative examples for the predicted class. Both are more valuable than weak labels for the purposes of fine-tuning a base model on locally relevant data. This is an active learning design, in which the system selectively accumulates the most informative training examples from its own deployment population, rather than relying on a fixed static dataset collected prior to deployment. The History screen is the front-end of that pipeline; the back-

end — a label management system, a retraining trigger, and a model update delivery mechanism — is the architectural work required in Version 2.

[SCREENSHOT]

Insert the History screen here — the chronological scan log showing disease names, dates, thumbnails, and confidence badges, as it appears in your Figma file.

Figure 4: History screen — user-facing diagnostic log and underlying data collection layer for model retraining.

4.5 Training Data Bias and the Domain Shift Problem in West African Agricultural Contexts

The training data bias problem is the most technically significant challenge facing Crop Care, and it is one that the interface cannot resolve on its own. It operates at the level of the ML pipeline, and its consequences are felt at the level of the user: a model that performs poorly on the specific diseases, crop varieties, and visual conditions present in the deployment context will return incorrect diagnoses with high confidence, which is both harmful to the farmer and destructive of trust in the application.

The domain shift between existing public datasets and the Crop Care deployment context has several distinct components. The visual domain shift is the most immediately apparent: PlantVillage images are photographed against plain backgrounds in controlled lighting, while field images are taken against complex natural backgrounds — soil, other plants, the farmer's hand — in variable outdoor lighting that can range from direct equatorial sunlight to heavy overcast within a single field visit. Convolutional neural networks trained on clean-background images learn features that partially encode background characteristics; when the background distribution changes substantially, classification accuracy degrades in ways that are not predicted by held-out test set performance on the original dataset.

The biological domain shift is less visible but equally significant. Maize leaf rust caused by *Puccinia sorghi* in Ghana presents with morphological characteristics that differ from the same disease in Iowa, because the pathogen strains, the host crop varieties, the temperature and humidity conditions, and the co-infection patterns differ between the two environments. A model trained on images of North American *Puccinia sorghi* may fail to recognize Ghanaian *Puccinia sorghi* presentations, or may confuse them with unrelated diseases whose visual appearance is closer to the training distribution. This is not a failure of the model — it is a fundamental consequence of the distributional mismatch between training data and deployment context.

The diseases most relevant to Crop Care's target context — maize lethal necrosis, cassava brown streak disease, cassava mosaic disease, and the specific rust and spot strains affecting West African crop varieties — are substantially underrepresented in all publicly available datasets. Cassava brown streak disease, for example, which causes significant losses in West and Central African cassava production, is absent from PlantVillage and has limited representation in the literature. A production model for Crop Care cannot be built from public datasets alone; it requires either a targeted data collection program in the deployment region, a partnership with an agricultural research institution

with access to locally annotated image libraries, or the community data contribution pipeline described in Section 4.4.

The confidence score on the result screen is the interface's primary tool for signaling to the user when a diagnosis may be unreliable, but it is only effective if the model's uncertainty estimate is calibrated against the true distribution of deployment inputs — not against a held-out test set drawn from the same controlled-condition training distribution. A model that is well-calibrated on PlantVillage test images may be systematically overconfident on field images from Ghana, because the field images are out-of-distribution relative to the training data. Addressing this requires either temperature scaling or Platt calibration applied after training on a representative set of locally collected field images, or a Bayesian inference approach that provides principled uncertainty estimates across the full input distribution. Either approach requires locally collected data — which returns the analysis to the data collection infrastructure of the History screen and the community contribution pathway.

5. Screen Walkthroughs

This section presents each screen group in the prototype alongside the corresponding design rationale. The framing here is more descriptive than in Section 4, intended to support review of the prototype alongside the analytical discussion above.

Screen 01 — Onboarding

The onboarding flow introduces the application to first-time users through a minimal two-screen sequence that establishes the core interaction model — photograph a leaf, receive a diagnosis — without requiring account creation or extended setup. The deliberate minimalism of the onboarding is motivated by adoption research showing that every additional step in an onboarding flow produces a measurable drop-off in completion rate, and that users with lower digital literacy are disproportionately affected by multi-step onboarding processes. The application is functional from the moment the user completes onboarding without any account-linked state, reducing the barrier to first use to the minimum necessary.

[SCREENSHOT]

Insert the Onboarding screens here — the introductory slides establishing the two-step process, arranged as they appear in your Figma file.

Figure 5: Onboarding screens — minimal introduction to the Crop Care interaction model.

Screen 02 — Home and Camera

The camera screen is described in detail in Section 4.3. In addition to the input guidance system discussed there, the dual-action design — Take Photo and Choose From Gallery — addresses an asymmetry between when a farmer notices a disease and when they have the opportunity to photograph it properly. A farmer who noticed symptoms at dawn but is now in a village with better light or more time should not need to return to the field to submit a photograph. The gallery option also creates a pathway for farmers to document

disease progression over time by photographing the same plant across multiple sessions and submitting each image for diagnosis.

[SCREENSHOT]

Insert the Camera screens here — all viewfinder state variants with guidance prompts and action buttons visible, as they appear in your Figma file.

Figure 6: Camera screens — input acquisition interface with embedded guidance and dual capture options.

Screen 03 — Analyzing and Result

The analyzing screen and result screen are described in detail in Sections 4.1 and 4.2. The result screen's progressive disclosure architecture — primary diagnosis and confidence score visible without scrolling, treatment and prevention steps accessible by scrolling, extended treatment detail available through a secondary expansion — is designed to serve both the user who needs a fast diagnosis to act on immediately and the user who wants to read the full treatment protocol carefully before purchasing inputs. The information hierarchy is established through typographic weight and spacing rather than color alone, ensuring legibility across color vision deficiencies and in the variable outdoor lighting conditions of field use.

[SCREENSHOT]

Insert the Analyzing and Result screens here — the processing state, the full result view with confidence score and treatment steps, and the expanded treatment detail, arranged in a single row as they appear in your Figma file.

Figure 7: Analyzing and Result screens — inference loading state and structured output presentation.

Screen 04 — History

The History screen is described in detail in Section 4.4. The calendar icon in the top right corner of the screen signals future date-range filtering functionality, intended primarily for seasonal review — a farmer comparing disease incidence across two growing seasons, or an extension officer reviewing the disease history for a cluster of farms they support. The calendar icon is included in the prototype as a forward-looking design signal rather than a functional element, establishing the intended trajectory of the feature without requiring its implementation within the current project scope.

[SCREENSHOT]

Insert the History screen here — the scan log with disease names, dates, thumbnails, and confidence badges, as it appears in your Figma file.

Figure 8: History screen — diagnostic log and data collection layer.

Screen 05 — Language

The language screen supports thirteen languages in the current prototype: English, Spanish, Hausa, Twi, Ga, Dangme, Akan, Adangme, Japanese, German, Italian, Danish,

and Finnish. Each language is displayed in both English and its native script. The decision to display native scripts is particularly significant for the Ghanaian language group: a user who cannot read English but can read Twi in its Roman-script orthography can identify their language without assistance. Language switching is persistent — the selected language is stored to the device and applied globally on every subsequent session without requiring re-selection. Treatment steps and disease names are localized in full, not transliterated; the clinical accuracy of the local-language treatment content must be verified by agricultural and linguistic experts before deployment.

[SCREENSHOT]

Insert the Language screen here — the full language list with native scripts, as it appears in your Figma file.

Figure 9: Language screen — persistent multilingual localisation with native script display.

6. Broader HCI Considerations

6.1 Trust Calibration in Human-AI Decision Systems

The Crop Care interface is, at its core, a trust calibration problem. A user who overtrusts the AI — who treats every output as correct regardless of confidence score — will apply wrong treatments on misclassified diagnoses with high confidence. A user who undertrusts the AI — who treats the application's output as no more reliable than a guess — will not act on correct diagnoses and will not benefit from the system at all. The interface aims to produce appropriately calibrated trust: a user who treats high-confidence outputs as reliable guides for action and low-confidence outputs as starting points for further verification.

This is a known challenge in human-AI teaming research (Parasuraman & Riley, 1997; Lee & See, 2004). Users tend to overtrust systems that are presented as authoritative and undertrust systems whose limitations are made visible. The Crop Care interface makes limitations visible through the confidence score, the Retake button, and the absence of absolute language in the result text. Whether this is sufficient to produce calibrated trust in the target user population is an empirical question that requires field testing — and one that cannot be answered by the interface design alone, because trust calibration is also a function of the user's prior experience with AI systems, their domain expertise, and the social context in which they encounter the tool.

6.2 Designing for the Edge of Connectivity

Connectivity-aware design — building systems that degrade gracefully as network availability decreases — is an underexplored area in HCI for emerging markets. Most mobile application design assumes a baseline of reliable connectivity and treats offline states as exceptions. For Crop Care, the offline state is the norm: the primary use case occurs in a field environment where cellular coverage is unreliable, data is expensive relative to income, and the farmer is unlikely to have a Wi-Fi connection. The interface design currently accommodates this through the gallery option — which allows users to

photograph disease symptoms when convenient and submit them for analysis later — but a production system requires the on-device inference architecture described in Section 4.2 to make the core identification function genuinely offline-capable.

The History screen provides offline value independently of inference: a farmer can review past diagnoses, re-read treatment steps, and compare symptoms across seasons without any connectivity. This offline utility of the history layer is an underappreciated aspect of the data architecture — it transforms the History screen from a passive log into an active reference tool that serves the user in precisely the conditions where connectivity-dependent features are unavailable.

6.3 Cognitive Load and Progressive Disclosure

The result screen is the most information-dense view in the application and presents the greatest cognitive load challenge. A farmer who has just photographed a diseased leaf and received a result is simultaneously managing the relief or anxiety of the diagnosis, the need to understand the treatment steps, the need to evaluate whether to trust the confidence score, and the decision of whether to act immediately or seek additional information. Presenting all of this information simultaneously — disease name, confidence score, five treatment steps, four prevention tips, and the source citation — risks overwhelming the user and reducing the probability that they retain and act on the most critical information.

The progressive disclosure architecture of the result screen is designed to reduce this cognitive load by presenting information in layers of increasing depth. The first layer — disease name and confidence score — is visible immediately and is sufficient for a user who simply needs to know what is wrong and how confident the AI is. The second layer — How To Treat It — is accessible by scrolling and provides the actionable treatment protocol. The third layer — extended treatment detail — is accessible by tapping an expansion affordance and provides additional technical information for users who want it. This layering is grounded in Sweller's cognitive load theory (1988), which argues that instructional design should minimize extraneous cognitive load — the processing burden imposed by the format of information — so that available cognitive resources can be directed toward the germane processing of the content itself.

7. Future Architecture: Version 2 Priorities

7.1 On-Device Inference Pipeline

The Version 2 technical priority is the implementation of a full on-device inference pipeline. This requires: training or fine-tuning a convolutional neural network — MobileNetV2 or EfficientNet-B0 — on a disease classification dataset supplemented with locally collected West African field images; applying post-training quantization to produce a TFLite model for Android and a Core ML model for iOS; integrating the model into the application's inference module with appropriate memory management; and validating the quantized model's accuracy and calibration on a held-out test set drawn from field-captured images representative of the deployment context. The camera guidance system described in

Section 4.3 should be extended with pre-capture quality gating — lightweight heuristics for blur and lighting adequacy — to reduce the rate of low-quality inference calls.

7.2 Active Learning and Community Annotation

The Version 2 data infrastructure priority is the implementation of a voluntary annotation pathway on top of the existing History screen. Users who confirm a diagnosis — indicating that they applied the prescribed treatment and observed the expected outcome — generate hard-labeled positive examples. Users who correct a diagnosis — indicating the model was wrong and providing the correct disease name — generate hard-labeled negative examples and initiate an active learning signal. A label management backend should collect these annotations, filter for quality using inter-annotator agreement where multiple users have scanned the same disease presentation, and trigger model fine-tuning at defined intervals when sufficient new labeled data has accumulated. This pipeline converts the History screen from a static log into a living data collection system that continuously improves the model's performance on the specific diseases and visual conditions present in the deployment context.

7.3 Expanded Disease and Crop Coverage

The current prototype demonstrates the detection pipeline using maize diseases — Leaf Rust and Maize Leaf Spot. Version 2 should extend coverage to cassava, yam, tomato, pepper, and cocoa — the crops with the highest economic significance in Ghana's main agricultural regions. A crop selection input, either as a dedicated screen before image capture or as a post-result context input, would allow the model to apply crop-specific prior probabilities to its classification, improving accuracy for visually similar diseases that require different treatment interventions depending on the affected host plant.

8. Conclusion

Crop Care is a mobile application designed to make AI-based crop disease diagnosis genuinely accessible to smallholder farmers in West Africa. This paper has presented the application's design with attention to five technical problems: communicating probabilistic uncertainty to non-technical users through a calibrated confidence score and visual encoding; deploying an inference model on low-specification hardware under TensorFlow Lite or Core ML; designing an input acquisition system that produces AI-usable images from users with no understanding of the model's requirements; architecting the History screen as a foundation for active learning and community data contribution; and understanding the training data bias and domain shift problem that is the primary obstacle to the model generalizing reliably in a West African agricultural deployment context.

The interface decisions described here are not aesthetic choices. Each one is a response to a specific technical or cognitive constraint imposed by the AI pipeline, the hardware environment, the literacy and digital experience of the target user, or the distributional mismatch between available training data and the actual deployment context. The design of a system that deploys AI in a high-stakes decision environment for non-technical users

is fundamentally an HCI problem as much as an ML problem — and the quality of the outcome depends on how well those two disciplines are integrated.

The prototype demonstrates that the integration is achievable. The work of building it, validating it in the field, and improving the model it depends on remains ahead.

9. References

Food and Agriculture Organization of the United Nations. (2021). The impact of disasters and crises on agriculture and food security: 2021. FAO. <https://www.fao.org/documents/card/en/c/cb3673en>

Ghana Statistical Service. (2021). Ghana 2021 Population and Housing Census: General Report. Ghana Statistical Service. <https://census2021.statsghana.gov.gh>

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, 70, 1321–1330.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint*. <https://arxiv.org/abs/1511.08060>

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713.

Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 50–80.

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419. <https://doi.org/10.3389/fpls.2016.01419>

Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230–253.

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning (ICML 2019), 97, 6105–6114.

World Bank. (2023). Agricultural technology adoption in Sub-Saharan Africa: Barriers and opportunities. World Bank Group. <https://www.worldbank.org/en/topic/agriculture>